Assignment #1 Historical Ciphers

Language chose: HTML and JavaScript

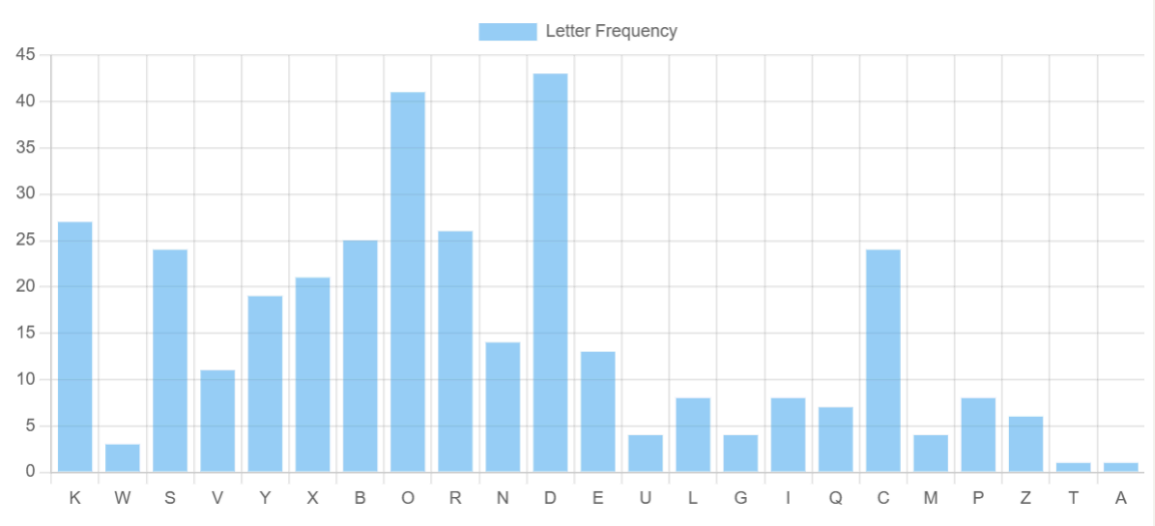
1. I finished this problem by using **frequency analysis attack**

   First I need to count each letter frequency in the ciphertext

```
/**
 * Count each letter frequency in a given ciphertext
 * @param {string} cipherText
 * @returns {object} key: letter value: frequency
 */
function countLetterFreq(ciphertext)
```

   This function helps me count frequency and store them in an object with key is letter and value is its corresponding frequency.

   Next is to draw each letter frequency of the ciphertext in a bar chart, I accomplished it by using a library called **chart.js**



   We can see the most frequent letters in ciphertext are E and O.

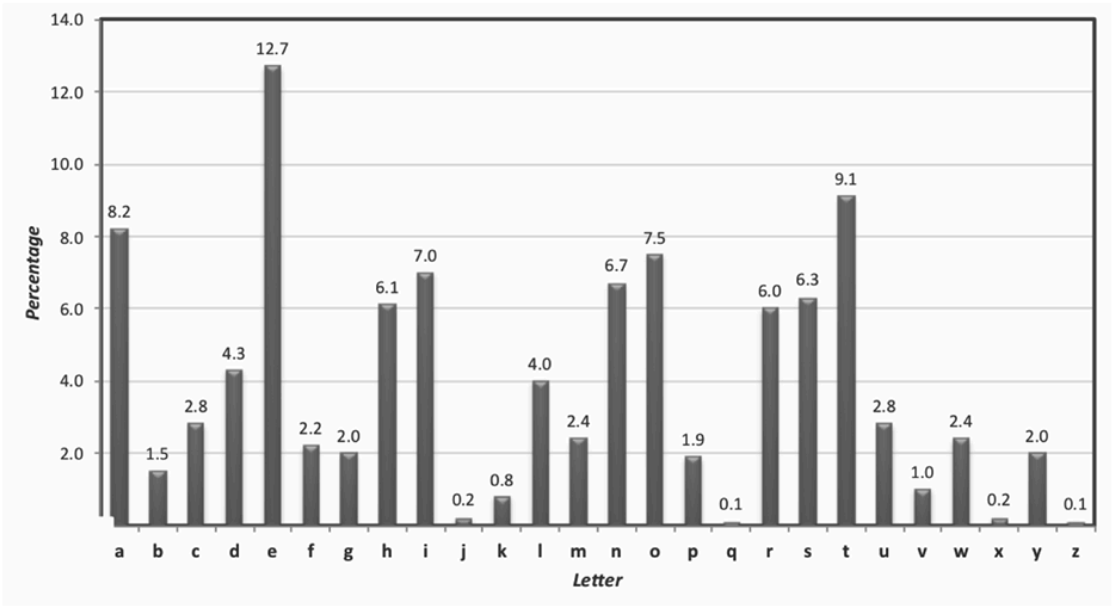


FIGURE 1.3: Average letter frequencies for English-language text.

By comparing the average letter frequencies for English text, I fist tried whether E(plain letter) encrypted into D(cipher letter).

This function helps me get the key in number based on the given plain letter and cipher letter.

```
/**
 * Get key number based on the given plain letter and cipher
letter
 * @param {char} plainLetter
 * @param {char} cipherLetter
 * @returns {number} 0-25
 */
function getKey(plainLetter, cipherLetter)
```

This function helps me get the possible plaintext based on the ciphertext and possible key

```
/**
 * decrypt ciphertext by left shifting each letter # of key
locations
 * @param {string} ciphertext
 * @param {number} key 0-25
 * @returns {string} possible plaintext
 */
function leftShiftByKey(ciphertext, key)
```

Try whether E(plain) -> D(cipher)

```
Key: 25 Plaintext:                                  RongJ a1.js:86
LXTWWTZYLTCPLSLCOSLELYOLOCFYVLCPLELMLCHSPYESPJRPEESPTC
MPPCDESPJYZETNPLQWJTYPLNSXFRESPXTWWTZYLTCPAZWTEPWJLDVD
ESPMLCEPYOPCQZCLYZESPCMPPCESPYACZNPPODEZDTATEESPSLCOSL
EDATWWDZFEUFDEPYZFRSEZRPECTOZQESPQWJLYOBFLQQDESPCPDETE
DYZHESPOCFYVDEFCYSPDETNVDSTDSLYOTYEZESPMPPCRCLMDESPQWJ
MJESPHTYRDLYODSZFEDDATETEZFEDATETEZFECPLOPCDOTRPDEQPMC
FLCJEHZESZFDLYOEPY
```

which does not seem correct as a plaintext

Try whether E(plain) -> O(cipher)

```
Key: 10 Plaintext:                                  RongJ a1.js:92
AMILLIONAIREAHARDHATANDADRUNKAREATABARWHENTHEYGETTHEIR
BEERSTHEYNOTICEAFLYINEACHMUGTHEMILLIONAIREPOLITELYASKS
THEBARTENDERFORANOTHERBEERTHENPROCEEDSTOSIPITTHEHARDHA
TSPILLSOUTJUSTENOUGHTOGETRIDOFTHEFLYANDQUAFFSTHERESTIT
SNOWTHEDRUNKSTURNHESTICKSHISHANDINTOTHEBEERGRABSTHEFLY
BYTHEWINGSANDSHOUTSSPITITOUTSPITITOUTREADERSDIGESTFEBR
UARYTWOTHOUSANDTEN
```

It is a normal plaintext starting with "A MILLIONAIRE"...

Therefore, the key is 10 and plaintext is above on the screenshot.

---

2. Decryption of Vigenère cipher

   First I need to get the key length by index of coincidence.

   Since the given ciphertext contains spaces, I need to remove them.

```
/**
 * remove any spaces and punctuation from a ciphertext
 * @param {string} ciphertext
 * @returns {string}
 */
function formatCiphertext(ciphertext)
```

My guess key length ranges from 1 to 10. Therefore, for each my guess key length k, I need to spilt the ciphertext into k groups and compute each group's index of coincidence. Then for each k, I calculate the average IC of all the groups. For the average that is closest to 0.065, that would be my best guess key length.

This is the helper function to calculate index of coincidence in a given text group

```
/**
 * Calculate IC in a given group
 * @param {string} text
 * @returns {number} IC
 */
function computeIC(text)
```

The formula to calculate is:

$$IC = \frac{\sum f_i(f_i - 1)}{N(N-1)}$$

where fi is each letter frequency and N is the length of the text

---

The function to calculate each key length's IC and store them in an object:

```
/**
 * Calculate average IC for each possible key length
 * @param {number} minK the minimum possible key length
 * @param {number} maxK the maximum possible key length
 * @param {string} ciphertext
 * @returns {object} key: key length, value: its average IC
 */
function avgICmap(minK, maxK, ciphertext)
```

```
1: 0.04376572040632425
2: 0.04864419191520075
3: 0.050408976137131474
4: 0.04730923085353465
5: 0.04863807677133959
6: 0.06397450126639916
7: 0.04452833987717708
8: 0.04696356275303644
9: 0.051846193022663606
10: 0.054717741935483 88
```

This the key length and its corresponding average IC map, in which when key

length is 6, the average IC is closest to 0.065.

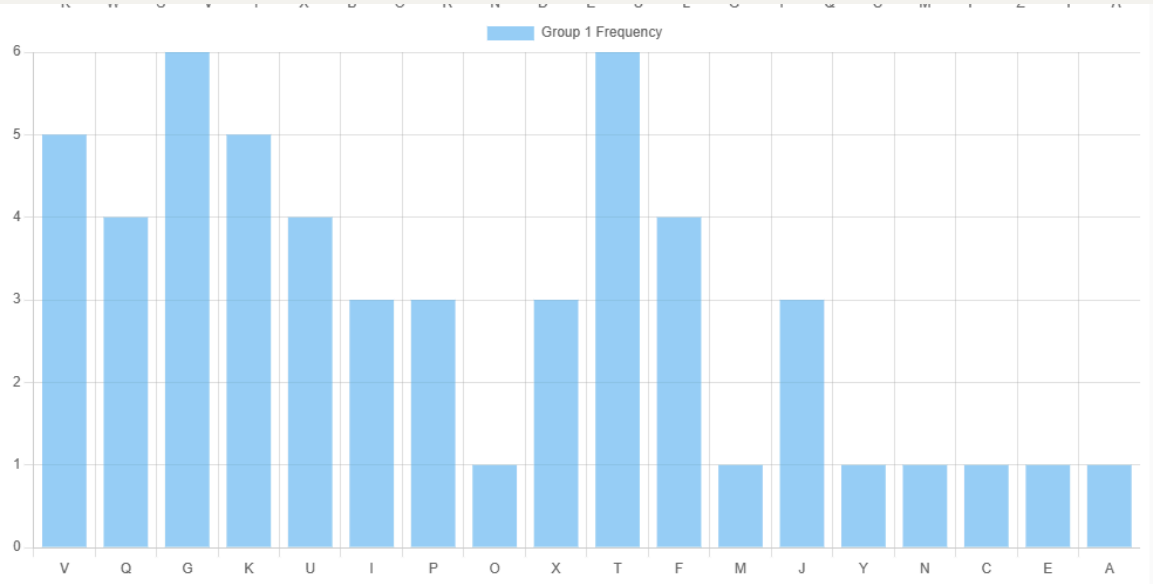
Therefore, the best guess key length is 6. Based on the key length, I spilt the ciphertext into 6 groups by taking every sixth letter.

```
0: "VQGVKUGGIGPOKXUTFMGPQXJUPYUVXTGIVNKTTVKFTCEIJFJKAFQQT"
1: "PVEMDALZMVOIKMWCXQPOVMPIAIQPQLJIPWHUMPVBMLIPCBMVAPVD"
2: "TSPCTDPTGTLVTCUVXTJHLGDCENMHCHJXTGTTPTTWAPAQGDEVWPTT"
3: "HAZAITFSVUPUUPJHUZULHDBKBMTALDZUDNKYAPZLHPUVYKYAVCPT"
4: "PVMEPXWSJHXMXRSXOXXEWMRKXSSLCICEIEJWIVEQGWIVMSYLYIRF"
5: "DVEKFYNEKZYWVXIVJYZJFKULRIEVRIRJCERKUMEFBZZJVKEVCUEV"
```

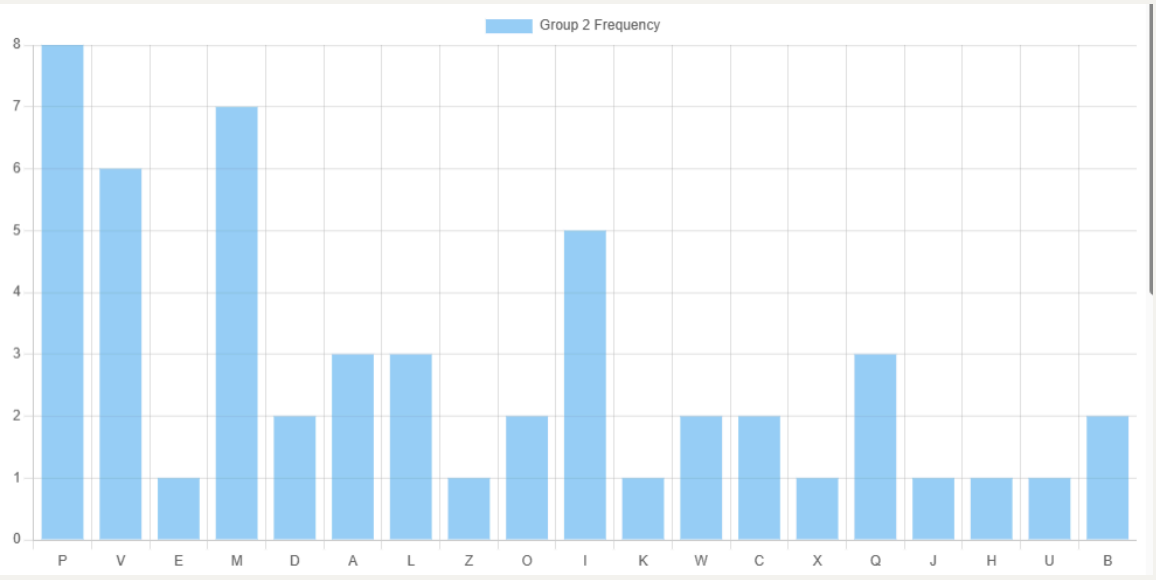
Each group was treated as a Caesar cipher, and I performed frequency analysis on each group like in question 1.

Letter frequency in each group

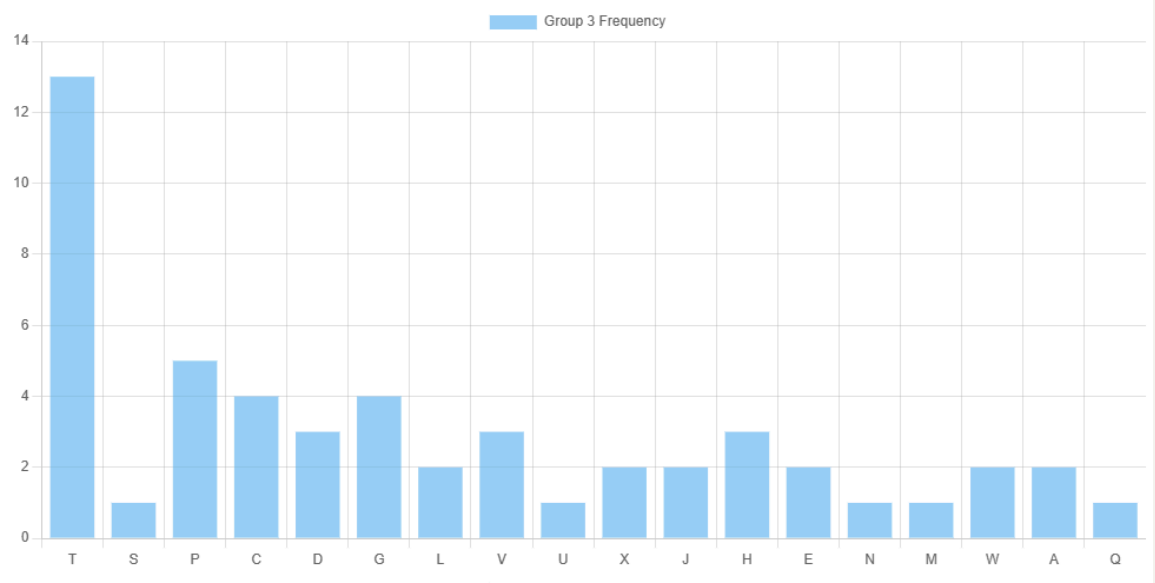
Group 1:



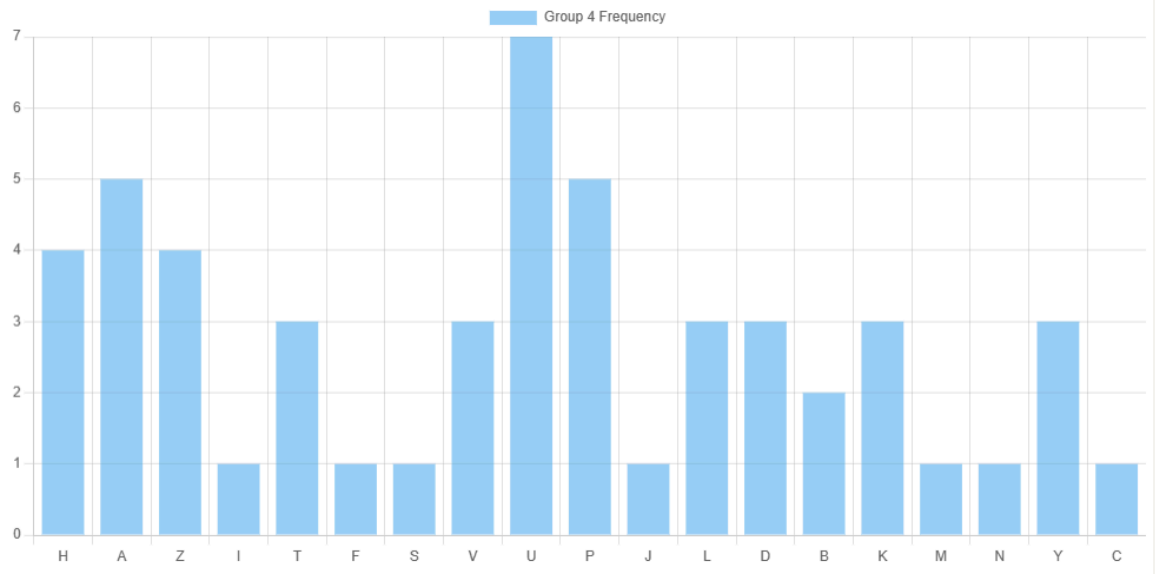
Group 2:



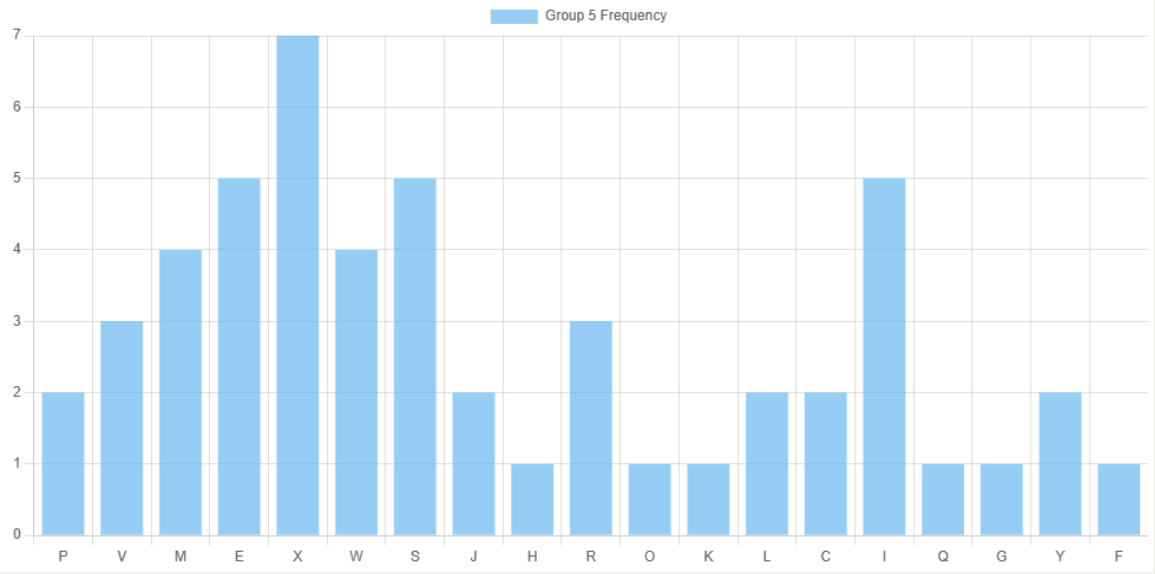
Group 3:



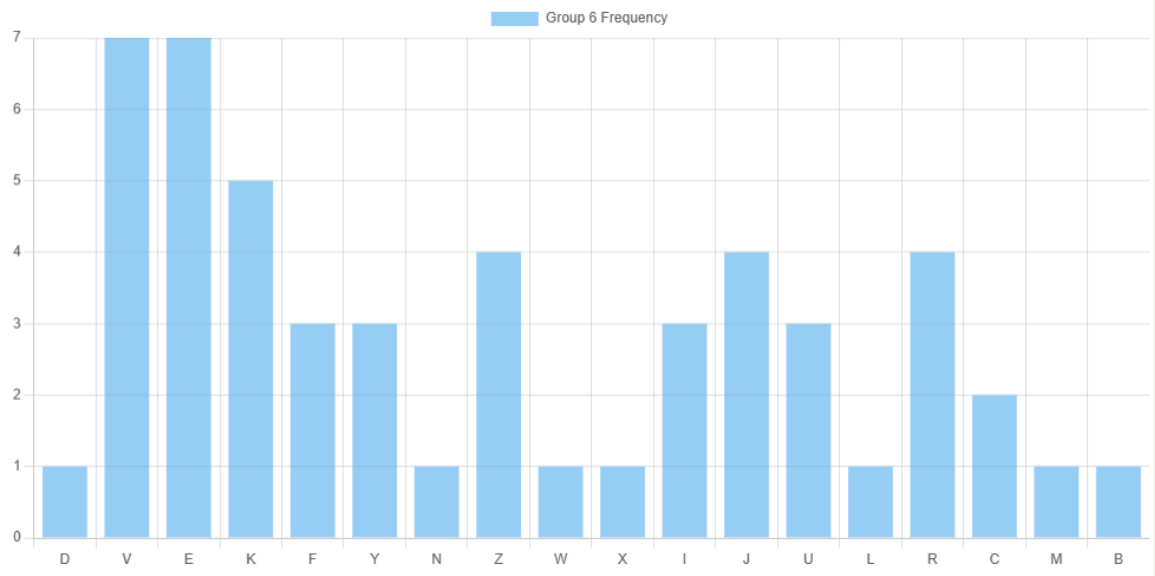
Group 4:



Group 5:



Group 6:



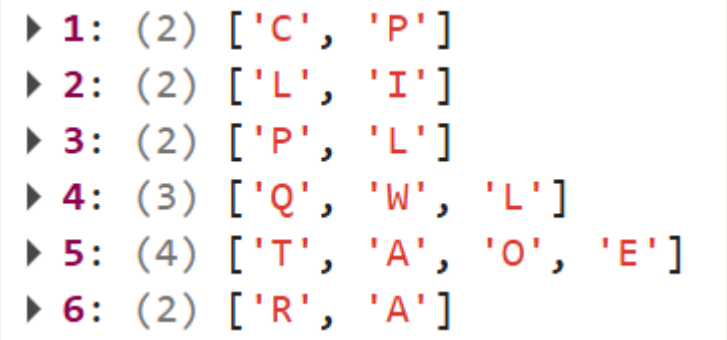
For each group, I identified the two most frequent ciphertext letters. I assumed that each of these letters could correspond to the letter E in English and computed the corresponding shifts. This produced two candidate key letters for each group.

Helper function to get letter key based on plain letter and cipher letter:

```
/**
 * Get key in letter based on plain letter and cipher letter
 * @param {string} plainLetter
 * @param {string} cipherLetter
 * @returns {string}
 */
function getLetterKey(plainLetter, cipherLetter)
```

PLAIN LETTER: E	GROUP 1	GROUP 2	GROUP 3	GROUP 4	GROUP 5	GROUP 6
POSSIBLE CIPHER LETTER	G, T	P, M	T, P	U, A, P	X, E, S, I	V, E

Here is the possible keys for each group:



One reasonable key is **cipher** even though the H does not exist in group 4.

Try CIPHER as the key:

Function to do Vigenère decryption:

```
/**
 * Vigenère decryption: P = (C - K) mod 26
 * @param {string} ciphertext
 * @param {string} key
 * @returns {string} plaintext
 */
function vigenereDecrypt(ciphertext, key)
```

The plaintext is below:

```
THEALMONDTREEWASINTENTATIVEBLOSSOMTHEDAYSWERELONGEROFTE RongJ a1.js:297
NENDINGWITHMAGNIFICENTEVENINGSOFCORRUGATEDPINKSKIESTHEHUNTINGSEASONWASO
VERWITHHOUNDSANDGUNSPUTAWAYFORSIXMONTHSTHEVINEYARDSWEREBUSYAGAINASTHEWE
LLORGANIZEDFARMERSTREATEDTHEIRVINESANDTHEMORELACKADAISICALNEIGHBORSHURR
IEDTODOTHEPRUNINGTHEYSHOULDHAVEDONEINNOVEMBER
```

It is clear that decryption succeeded with plaintext beginning with "The almond tree was..."

Therefore, the key is CIPHER, and the recovered plaintext is meaningful in English.

---

3. I finished this problem in Node.js platform since browser can not access files in our computer without user's permission.

   First I have the helper function to XOR each plaintext byte with the key byte to generate ciphertext bytes

```
/**
 * XOR plaintext every byte with key to generate an arrary
containing ciphertext bytes
 * @param {Array} bytes each element is 0-255
 * @param {number} key 0-255
 * @returns
 */
function xorEncHelper(bytes, key)
```

Then I have the encryption function to generate ciphertext in hex, and write ciphertext into output file:

The data flow of the encryption algorithm:

plaintext -> plaintext bytes -> each byte XOR with key -> ciphertext bytes -> ciphertext in hex

```
/**
 * XOR each byte of plaintext in a plaintext file with key
to produce ciphertext in hex
 * @param {*} plaintextFileName
 * @param {*} ciphertextFileName
 * @param {*} keyFileName
 */
function xorEncryption(plaintextFileName,
ciphertextFileName, keyFileName)
```

Finally I have the decryption function to recover plaintext in character from ciphertext in hex

The data flow of the decryption algorithm:

ciphertext in hex -> ciphertext bytes -> each byte XOR with key -> plaintext bytes -> plaintext

```
/**
 * XOR decrypt ciphertext file and recover plaintext
 * @param {string} ciphertextFileName file containing
ciphertext in hex
 * @param {string} outputPlaintextFileName file to write
recovered plaintext
 * @param {string} keyFileName file containing 2 hex symbols
 */
function xorDecryption(
  ciphertextFileName,
  outputPlaintextFileName,
  keyFileName,
)
```

Test case:

Original Plaintext:

CSC 768 > HW1 > plaintext.md

```
1  Principles of modern cryptography over the past several decades cryptography has developed into more of a science schemes
   are now developed and analyzed in a more systematic manner with the ultimate goal being to give a rigorous proof that a
   given construction is secure in order to articulate such proofs we first need formal definitions that pin down exactly
   what secure means such definitions are useful and interesting in their own right as it turns out most cryptographic proofs
   rely on currently unproven assumptions about the algorithmic hardness of certain mathematical problems any such
   assumptions must be made explicit and be stated precisely an emphasis on definitions assumptions and proofs distinguishes
   modern cryptography from classical cryptography we now discuss these three principles in greater detail
```

Key:

CSC 768 > HW1 > key.md

```
1  2E
```

Ciphertext:

CSC 768 > HW1 > ciphertext.md

```
1  7e5c47404d475e424b5d0e41480e43414a4b5c400e4d5c575e5a41495c4f5e46570e41584b5c0e5a464b0e5e4f5d5a0e5d4b584b5c4f420e4a4b4d4f4a4
   b5d0e4d5c575e5a41495c4f5e46570e464f5d0e4a4b584b42415e4b4a0e47405a410e43415c4b0e41480e4f0e5d4d474b404d4b0e5d4d464b434b5d0e4f
   5c4b0e4041590e4a4b584b42415e4b4a0e4f404a0e4f404f4257544b4a0e47400e4f0e43415c4b0e5d575d5a4b434f5a474d0e434f40404b5c0e59475a4
   60e5a464b0e5b425a47434f5a4b0e49414f420e4c4b4740490e5a410e4947584b0e4f0e5c4749415c415b5d0e5e5c4141480e5a464f5a0e4f0e4947584b
   400e4d41405d5a5c5b4d5a4741400e475d0e5d4b4d5b5c4b0e47400e415c4a4b5c0e5a410e4f5c5a474d5b424f5a4b0e5d5b4d460e5e5c4141485d0e594
   b0e48475c5d5a0e404b4b4a0e48415c434f420e4a4b484740475a4741405d0e5a464f5a0e5e47400e4a4159400e4b564f4d5a42570e59464f5a0e5d4b4d
   5b5c4b0e434b4f405d0e5d5b4d460e4a4b484740475a4741405d0e4f5c4b0e5b5d4b485b420e4f404a0e47405a4b5c4b5d5a4740490e47400e5a464b475
   c0e4159400e5c4749465a0e4f5d0e475a0e5a5b5c405d0e415b5a0e43415d5a0e4d5c575e5a41495c4f5e46474d0e5e5c4141485d0e5c4b42570e41400e
   4d5b5c5c4b405a42570e5b405e5c41584b400e4f5d5d5b435e5a4741405d0e4f4c415b5a0e5a464b0e4f4249415c475a4643474d0e464f5c4a404b5d5d0
   e41480e4d4b5c5a4f47400e434f5a464b434f5a474d4f420e5e5c414c424b435d0e4f40570e5d5b4d460e4f5d5d5b435e5a4741405d0e435b5d5a0e4c4b
   0e434f4a4b0e4b565e42474d475a0e4f404a0e4c4b0e5d5a4f5a4b4a0e5e5c4b4d475d4b42570e4f400e4b435e464f5d475d0e41400e4a4b484740475a4
   741405d0e4f5d5d5b435e5a4741405d0e4f404a0e5e5c4141485d0e4a475d5a4740495b475d464b5d0e43414a4b5c400e4d5c575e5a41495c4f5e46570e
   485c41430e4d424f5d5d474d4f420e4d5c575e5a41495c4f5e46570e594b0e4041590e4a475d4d5b5d0e5a464b5d4b0e5a465c4b4b0e5e5c47404d475
   e424b5d0e47400e495c4b4f5a4b5c0e4a4b5a4f4742
```

Recovered text:

CSC 768 > HW1 > recoverText.md

```
1  Principles of modern cryptography over the past several decades cryptography has developed into more of a science schemes
   are now developed and analyzed in a more systematic manner with the ultimate goal being to give a rigorous proof that a
   given construction is secure in order to articulate such proofs we first need formal definitions that pin down exactly
   what secure means such definitions are useful and interesting in their own right as it turns out most cryptographic proofs
   rely on currently unproven assumptions about the algorithmic hardness of certain mathematical problems any such
   assumptions must be made explicit and be stated precisely an emphasis on definitions assumptions and proofs distinguishes
   modern cryptography from classical cryptography we now discuss these three principles in greater detail
```